

A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA

Progetto Finale di

Signal Processing and Optimization for Big Data

Corso di Laurea Magistrale in Ingegneria Informatica e Robotica – A.A. 2024-2025

Dipartimento di Ingegneria

docente

Prof. Paolo Banelli

Federated Spam Classifier

Training di un classificatore per messaggi Spam in modo distribuito e privato applicando tecniche di Federated Learning

studente

Federico Ombelli

1 Introduzione

L'obiettivo di questo progetto è esplorare le tecniche e gli algoritmi di **Federated Learning**, applicandoli ad un problema reale di classificazione di messaggi di Spam.

Seguendo il paradigma tradizionale di machine learning, per addestrare un modello di classificazione di questo tipo sarebbe necessario prima di tutto raccogliere i dati, gli SMS ricevuti dagli utenti in questo caso, memorizzarli in un unico server per poi procedere ad un training centralizzato. Se da una parte questo approccio permette di ottenere generalmente una migliore efficacia nell'addestramento, dall'altra richiede tuttavia la raccolta massiva di dati personali sollevando criticità riguardanti la **privacy** degli utenti e possibili rischi di violazione dei dati.

In risposta a queste sfide, il **Federated Learning** è emerso negli ultimi anni come un approccio promettente per permettere l'addestramento di modelli di machine learning in modo **decentralizzato** e **distribuito** sui dispositivi, senza la necessità di centralizzare i dati.

In questo documento si procede quindi ad implementare, analizzare e confrontare diverse strategie di ottimizzazione federata applicate a un classificatore di messaggi spam basato su **Logistic Regression**.

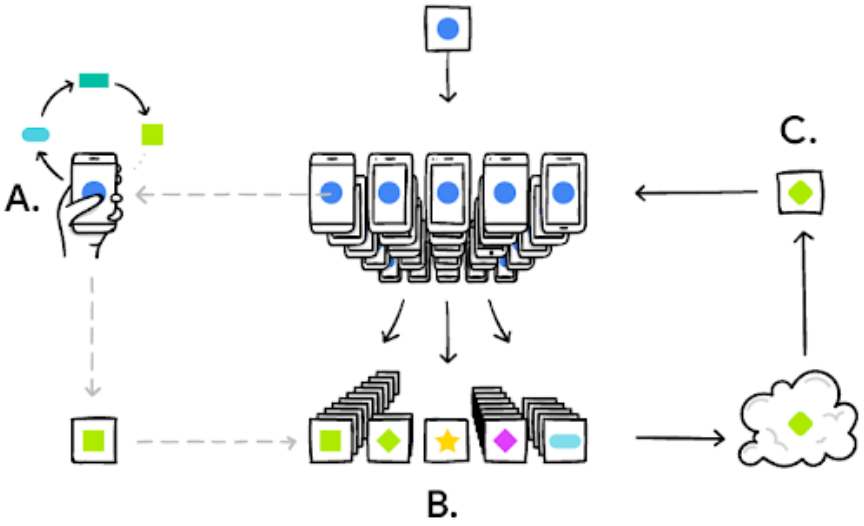


Figure 1.1: Schema semplificato del funzionamento del Federated Learning (fonte:Google Research)

1.1 Federated Learning

Il Federated Learning è una tecnica di machine learning distribuito introdotta da Google nel 2017 [3], progettata per addestrare modelli su dati memorizzati in modo decentralizzato sui dispositivi. A differenza dell'ottimizzazione distribuita tradizionale, dove il dataset è a disposizione del server centrale e viene partizionato su diverse macchine per parallelizzare il calcolo, in un contesto federato il modello viene addestrato localmente sui dispositivi (i client) utilizzando i dati generati in locale e solo gli aggiornamenti dei parametri (i gradienti o i pesi del modello) vengono condivisi con il server centrale.

Tipicamente il training di un modello tramite Federated Learning comprende 4 fasi principali:

- **Distribuzione del modello**: Il server invia il modello globale corrente w_t a un sottoinsieme di client selezionati
- **Training locale**: Ogni client k aggiorna il modello utilizzando i propri dati privati locali tipicamente tramite Stochastic Gradient Descent e calcola i nuovi pesi del modello w_{t+1}^k
- **Upload degli aggiornamenti**: I client inviano al server solo gli aggiornamenti del modello, mantenendo i dati privati
- **Aggregazione**: Il server aggrega gli aggiornamenti ricevuti per produrre il nuovo modello globale w_{t+1}.

1.1.1 Vantaggi Principali

L'adozione del Federated Learning offre vantaggi sostanziali rispetto all'apprendimento centralizzato:

- **Privacy-by-Design**: Poiché i dati non lasciano mai il dispositivo dell'utente, si riducono i potenziali rischi per la privacy e la sicurezza dei dati degli utenti
- **Efficienza di Comunicazione**: Invece di trasmettere i dati grezzi, vengono scambiati solo i parametri del modello, tipicamente di dimensioni molto inferiori rispetto ai dati di training.
- **Utilizzo di Dati Reali**: Permette di apprendere da dati che, per la loro natura sensibile o volume eccessivo, non possono essere caricati su un server, ad esempio in ambito sanitario.

2 Descrizione del problema

Il problema che si vuole risolvere è un problema di classificazione binaria supervisionata. Dato un insieme di messaggi, l'obiettivo è addestrare un modello in grado di mappare il contenuto testuale di un messaggio x in una classe $y \in \{0, 1\}$ dove 1 indica "spam" e 0 indica "non spam".

Nello scenario federato, il dataset totale $\mathcal{D}$ è partizionato su K client. Ogni client k possiede un dataset locale privato $\mathcal{D}_k$, con n_k campioni e $N = \sum_{k=1}^{K} n_k$ è il numero totale di campioni del dataset.

2.1 Il Modello implementato

Data la natura del problema è stato scelto un modello di **Logistic Regression** poiché permette di esplorare anche le tecniche di ottimizzazione convessa come ADMM in ambito federato.

Il modello di Logistic Regression stima la probabilità che un dato input x appartenga alla classe positiva (Spam) tramite la funzione sigmoide:

$$h_w(x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

Dove w è il vettore dei pesi e b è il bias. L'obiettivo del training è minimizzare la funzione di Loss che nella Logistic Regression, è dato dalla **Binary Cross-Entropy**.

$$\ell(w; x_i, y_i) = -\left[y_i \log(h_w(x_i)) + (1 - y_i) \log(1 - h_w(x_i))\right]$$

La **funzione di costo globale** è data dalla media della loss su tutti gli N campioni del dataset globale:

$$f(w) = \frac{1}{N} \sum_{i=1}^{N} \ell(w; x_i, y_i) + \frac{\lambda}{2} ||w||^2$$

È stato incluso anche un termine di regolarizzazione L2 ($\frac{\lambda}{2}||w||^2$) per prevenire l'overfitting sui dati locali.

La funzione di costo globale è additiva ed è quindi possibile riscriverla raggruppando in base al client, possiamo quindi eseguire uno *split-by-examples* del problema, suddividendolo in sottoproblemi ciascuno relativo a un sottoinsieme locale del dataset. La **funzione di costo locale** per ogni client k è quindi definita come:

$$f_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{D}_k} \ell(w; x_i, y_i) + \frac{\lambda}{2} ||w||^2$$

Problema di Ottimizzazione globale

Il problema di ottimizzazione globale è formulato come la minimizzazione della somma delle funzioni di costo locali rispetto ai pesi del modello w

$$\min_w f(w) = \min_w \sum_{k=1}^{K} f_k(w)$$

2.2 Algoritmi Implementati

Per risolvere il problema di ottimizzazione descritto sopra, sono stati implementati e confrontati tre diversi algoritmi iterativi **FedAvg**, **FedProx** e **FedADMM**.

FedAvg

FedAvg (*Federated Averaging*) è stato il primo algoritmo introdotto per il Federated Learning. Il training è suddiviso in round, ad ogni round il modello globale viene inviato ai K client o a un sottoinsieme di essi.

- Ciascun client esegue più epoche di *Stochastic Gradient Descent* (SGD) sui propri dati locali, partendo dal modello globale corrente w_t.
- Il server aggrega gli aggiornamenti ricevuti dai client calcolando la media pesata dei pesi ricevuti: $w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{N} w_{t+1}^k$.

Questo algoritmo è stato introdotto da Google nel 2017 ed è stato utilizzato con successo ad esempio per il training di modelli per la previsione del testo sulla *GBoard*, la tastiera usata sulla maggior parte degli smartphone Android [2]. Nel paper originale [3] si dimostra che eseguire più aggiornamenti di SGD sui client per ogni round permette di ottenere performance migliori con meno round di comunicazione rispetto ad eseguire un singolo step di gradiente locale ad ogni round. Sebbene efficiente, FedAvg può non garantire la convergenza ottimale quando i dati sono fortemente Non-IID.

FedProx

FedProx (*Federated Proximal Optimization*) è stato introdotto per affrontare l'eterogeneità dei dati [4]. Modifica il problema di ottimizzazione locale aggiungendo un termine prossimale alla funzione di costo:

$$\min_w \left(f_k(w) + \frac{\mu}{2} ||w - w_{global}||^2 \right)$$

Il parametro μ controlla quanto il modello locale può discostarsi dal modello globale w_t limitando le forti variazioni locali in caso di dati non distribuiti in modo statisticamente uniforme (Non-IID) e rendendo l'aggiornamento più stabile. Durante il training locale lo step di gradiente diventa:

$$w \leftarrow w - \eta \left(\nabla f_k(w) + \mu(w - w_{global}) \right)$$

FedADMM

FedADMM (*Federated Alternating Direction Method of Multipliers*) applica l'algoritmo di ottimizzazione **ADMM** al contesto federato, scomponendo il problema globale in sottoproblemi. A differenza di FedAvg che usa una media euristica, FedADMM introduce variabili duali (moltiplicatori di Lagrange) per coordinare il consenso. Il problema di ottimizzazione formulato per ADMM diventa quindi:

$$\min_{w_k, z} \sum_{k=1}^{K} f_k(w_k) + g(z) \quad \text{subject to} \quad w_k - z = 0$$

In cui si creano K copie locali w_k della variabile di ottimizzazione w e si impone il consenso con la variabile globale z.

- $g(z)$ è un termine di regolarizzazione applicato in modo globale ad esempio una norma L1.
- $f_k(w_k)$ è la funzione di costo locale che comprende la loss e la regolarizzazione L2 come indicato nella descrizione del problema.

La Lagrangiana Aumentata dell'ADMM in forma scalata è:

$$L_\rho(w, z, u) = \sum_{k=1}^{K} \left(f_k(w_k) + \frac{\rho}{2} \|w_k - z + u_k\|_2^2 - \frac{\rho}{2} \|u_k\|_2^2 \right)$$

Dove u_i è la variabile duale scalata (rappresenta l'accumulo degli errori di consenso). L'algoritmo alterna tre passaggi:

1. **Primal Update (Locale)**: Ogni client minimizza la Lagrangiana aumentata $L_\rho(w, z, u)$ rispetto alla variabile locale w_k

   $$w_k^{t+1} = \arg\min_{w_k} \left(\{_k(w_k) + \frac{\rho}{2} \|w_k - z^t + u_k^t\|_2^2 \right)$$

   Poiché non esiste una soluzione chiusa per la risoluzione si usa il Gradient Descent (SGD).

2. **Global Consensus (Server)**: si minimizza rispetto alla variabile globale z, minimizzando quindi il termine di regolarizzazione globale L1 e il termine di penalità della lagrangiana

$$z^{t+1} = \arg\min_{z} \left(\lambda_{L1} \|z\|_1 + \sum_{k=1}^{K} \frac{\rho}{2} \|w_k^{t+1} - z + u_k^t\|_2^2 \right)$$

Questo corrisponde ad eseguire sul server la media delle variabili primali w_k e duali u_k a cui si applica l'operatore di Soft Thresholding per la regolarizzazione L1 globale:

$$\bar{z} = \frac{1}{K} \sum_{k=1}^{K} (w_k^{t+1} + u_k^t)$$

$$z^{t+1} = S_t(\bar{z}) \quad \text{con soglia } t = \frac{\lambda_{L1}}{K \cdot \rho}$$

3. **Dual Update (Locale)**: I client aggiornano le proprie variabili duali u_k basandosi sullo scostamento tra il loro modello locale w_k e il modello globale z

$$u_k^{t+1} = u_k^t + (w_k^{t+1} - z^{t+1})$$

3 Implementazione

3.1 Dataset

Per simulare un problema reale si è scelto di utilizzare il dataset **UCI SMS Spam Collection**, molto utilizzato per problemi di classificazione di spam. Il dataset contiene 5574 messaggi SMS reali in inglese etichettati come *Spam* o *Ham* (non spam) raccolti da diverse fonti.

3.2 Tecnologie Utilizzate

Gli algoritmi descritti sopra sono stati implementati in **Python**, sfruttando librerie standard per il calcolo e l'analisi dati, come **Numpy**, **Pandas** e **Scikit-learn**. Si è scelto di implementare gli algoritmi federati e lo Stochastic Gradient Descent locale manualmente con operatori Numpy senza utilizzare framework di alto livello come *TensorFlow*, in modo da avere il controllo completo sulle operazioni di calcolo dei gradienti e di aggiornamento utile in particolare per l'implementazione di ADMM. Scikit-learn è stata impiegata per la vettorizzazione dei dati e per il calcolo delle metriche di valutazione.

3.3 Simulazione dell'ambiente federato

Per simulare la rete di dispositivi sui cui applicare gli algoritmi di Federated Learning si è scelto di implementare una strategia semplificata eseguendo ogni round di addestramento sui client virtuali in modo sequenziale su un unica macchina locale ed eseguendo l'aggregazione alla fine.

Per simulare la distribuzione dei dati il dataset è stato inizialmente suddiviso in una parte pubblica riservata al server e una privata distribuita tra i client e non condivisa con il server. Il set riservato al server è stato diviso in tre parti: un set usato per la creazione del dizionario, un validation set e un test set. Il set privato è stato suddiviso e distribuito tra i client. Per simulare la variabilità statistica dei dati (Non-IID), il dataset è stato distribuito variando in modo casuale sia la quantità di campioni sia lo sbilanciamento delle classi tra i vari client. Per ottenere questo sono state usate le distribuzioni di Dirichlet con parametro α variabile che permette di controllare lo sbilanciamento dei dati.

3.4 Preprocessing dei dati

Per applicare algoritmi di classificazione come la Logistic Regression a dati testuali (in questo caso messaggi SMS), è necessario convertire il testo in vettori numerici a dimensione fissa. In un contesto di training centralizzato si userebbero delle tecniche di vettorizzazione come **Bag-of-Words** e **TF-IDF**. BoW richiede la costruzione di un vocabolario globale delle possibili parole e mappa ogni messaggio in un vettore di features in cui ogni indice corrisponde a una possibile parola (feature) e il cui valore è dato dal numero di occorrenze della parola nel messaggio. Il TF-IDF normalizza i valori delle frequenze mitigando l'impatto delle parole troppo comuni nel dataset.

Tuttavia in un contesto federato il server non può accedere a tutto il dataset quindi non può applicare direttamente questo approccio.

- Una possibile soluzione è basata sull'uso di un **dataset pubblico**: il server costruisce il dizionario basandosi sui dati pubblici a disposizione, genera il vettorizzatore e lo distribuisce ai client che lo applicano direttamente ai propri dati locali per trasformarli.

- Un'altra soluzione è utilizzare un **Hashing Vectorizer** che usa una funzione di hash fissa per mappare le parole in indici. Rispetto alla soluzione precedente non serve un dizionario centrale poiché due client mapperanno una stessa parola nello stesso indice, tuttavia potrebbe generare delle collisioni se si sceglie un numero di feature troppo basso.

Per questo progetto si è scelto di implementare l'approccio con dataset pubblico poiché è quello che spesso viene usato in applicazioni reali di Federated Learning come ad esempio da Google per la previsione del testo nella tastiera *GBoard* [2].

4 Risultati

4.1 Training centralizzato

Per avere una *baseline* per le performance di classificazione del modello di Logistic Regression prima di tutto è stato addestrato il modello in modo centralizzato. Per eseguire un confronto equo con gli algoritmi di Federated Learning è stato usato lo stesso vettorizzatore generato con il dataset pubblico. È stato usato inoltre lo *Stochastic Gradient Descent* come algoritmo di ottimizzazione, tramite la classe *SGDCLassifier* di *scikit-learn* impostando la loss di tipo logistico, la regolarizzazione L2 e una learning rate costante.

Sono stati quindi testati diversi parametri di regolarizzazione e learning rate valutandone le performance di F1Score sul validation set.

I parametri che hanno ottenuto i migliori risultati su 300 epoche sono stati i seguenti: L2: 0.0001; learning rate: 1.0

F1 score su validation set: 0.978

Performance su test set:

- Accuracy: 0.98
- Precision: 0.96
- Recall: 0.92
- F1-Score: 0.94

4.2 Training Federato

Per valutare le performance dei tre algoritmi federati si è scelto prima di ottimizzare gli iperparametri per ciascun algoritmo eseguendo una grid search sul validation set per 100 round. Una volta trovate le configurazioni migliori si esegue un confronto tra diversi algoritmi per valutare le performance al variare della distribuzione dei dati tra i client e del numero di round.

Per simulare l'ambiente federato e la distribuzione Non-IID dei dati, inizialmente si è impostato un parametro di *alpha = 0.5* per la variabilità dei dati tra i client, ottenendo la distribuzione seguente su 10 client:

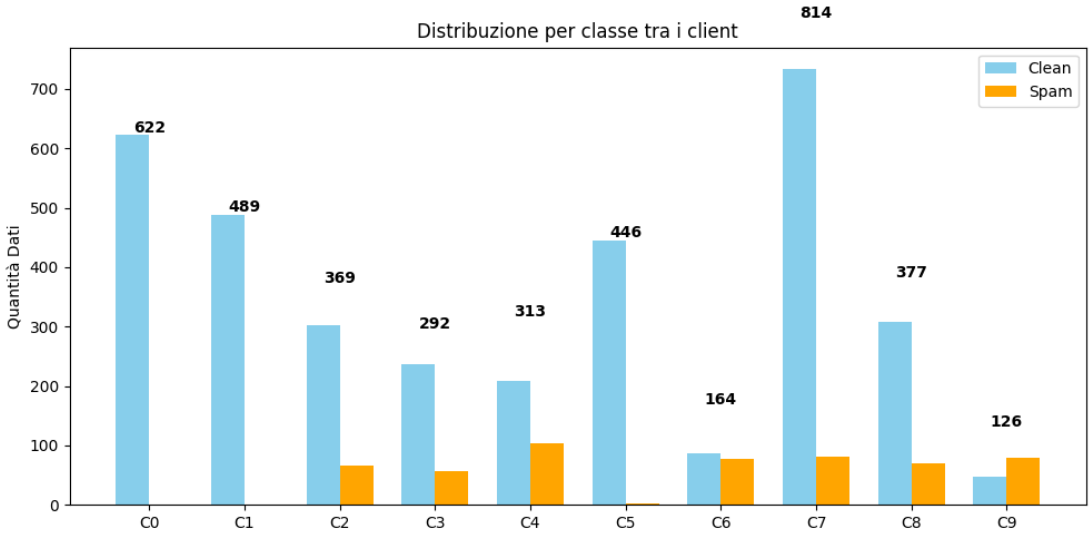


Figure 4.1: Distribuzione dei dati Non-IID dei dati tra i client con alpha=0.5

Algoritmo FedAvg

Per ricercare la configurazione migliore per l'algoritmo FedAvg è stata eseguita una *grid search* sul validation set per i parametri più critici, la **learning rate** e il numero di **epoche** di SGD locali. Le configurazioni testate sono:

- learning rate = [0.5, 1.0, 2.0, 3.0]
- local epochs = [5, 10, 20]

Per limitare la ricerca sono stati invece fissati in modo empirico:

- batch size = 64
- λ L2= 0.001

La migliore configurazione dopo 100 round di training è stata quella con **local epochs = 20** e **learning rate = 3.0**, che ha ottenuto i seguenti valori delle metriche sul validation set:

- Accuracy: 0.98
- Precision: 0.98
- Recall: 0.86
- F1-Score: 0.92

- Loss: 0.07

Confrontando le curve delle metriche al variare del numero di round si nota che l'algoritmo va in convergenza più velocemente al crescere del numero di epoche locali e della learning rate, come si può vedere nelle figure 4.2 e 4.3.

Algoritmo FedProx

Per ricercare la configurazione migliore per l'algoritmo FedProx è stata eseguita in modo analogo una *grid search* sul validation set per i parametri **learning rate** e il **parametro prossimale** μ Le configurazioni testate sono:

- learning rate = [0.1, 0.5, 1.0, 2.0]
- μ = [0.001, 0.01, 0.1, 1.0]

Per limitare la ricerca sono stati invece fissati:

- batch size = 64
- λ L2= 0.001
- local epochs = 20

La migliore configurazione dopo 100 round di training è stata quella con **learning rate = 2.0** e μ **= 0.001**, che ha ottenuto valori delle metriche molto simili a quelli ottenuti con FedAvg.

- Accuracy: 0.98
- Precision: 0.98
- Recall: 0.84
- F1-Score: 0.91
- Loss: 0.08

Da questi risultati si può ipotizzare che il modello Logistic Regression, per questa configurazione di eterogeneità dei dati (**alpha = 0.5**), non beneficia della correzione prossimale di FedProx, convergendo infatti verso valori di μ bassi. Confrontando le curve delle metriche al variare del numero di round notiamo inoltre che FedProx diverge con combinazioni di learning rate e μ alti (vedi figura 4.4).

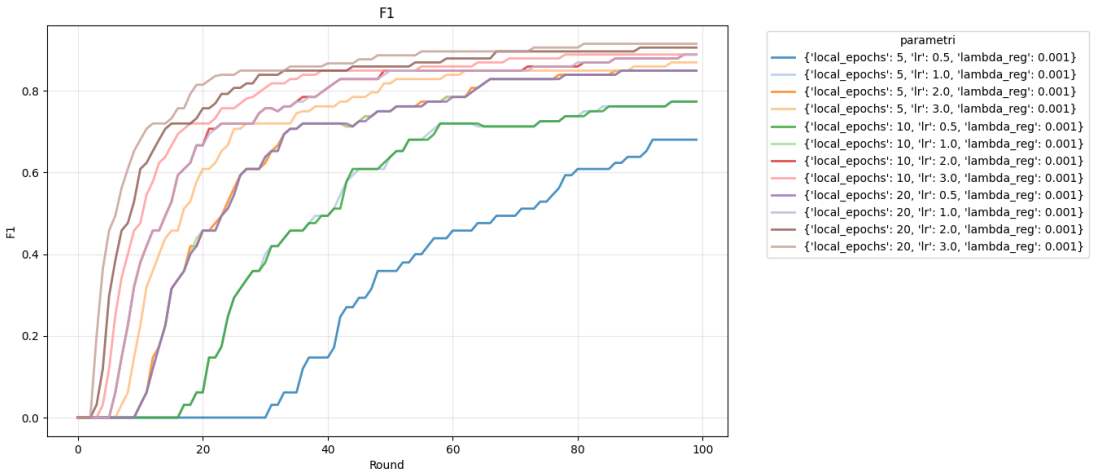


Figure 4.2: Andamento di F1 al variare dei round per l'algoritmo FedAvg

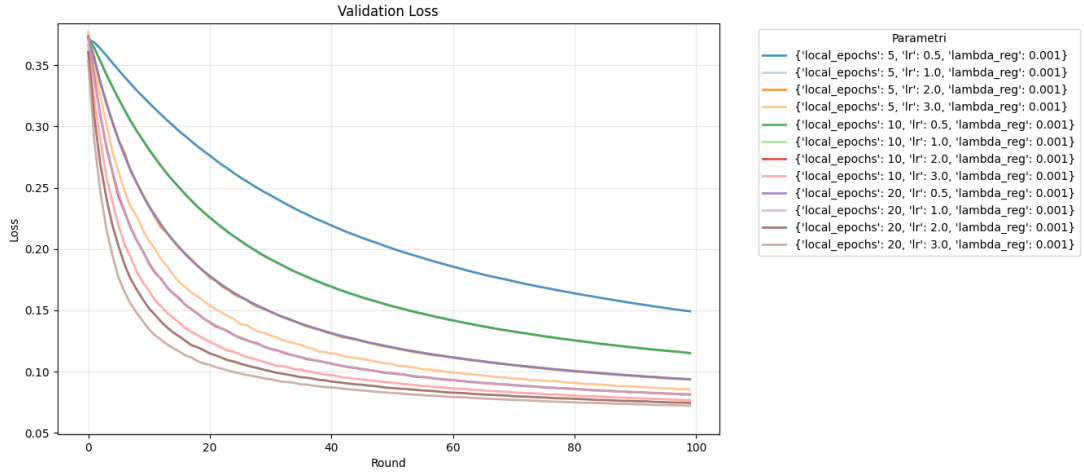


Figure 4.3: Andamento della Loss al variare dei round per l'algoritmo FedAvg

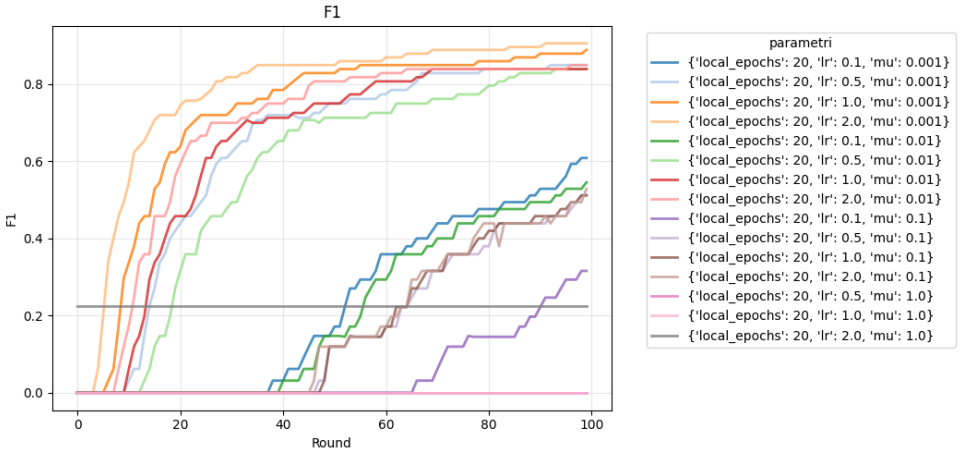


Figure 4.4: Andamento di F1 al variare dei round per l'algoritmo FedProx

Algoritmo FedADMM

Infine per l'algoritmo FedADMM sono stati esplorati in questo caso i parametri di **learning rate** e il **parametro di penalità** ρ Le configurazioni testate sono:

- learning rate = [0.1, 0.5, 1.0, 2.0]
- ρ = [0.01, 0.1, 1.0, 2.0]

Per limitare la ricerca sono stati invece fissati:

- batch size = 64
- λ L2= 0.001
- λ L1 globale = 0.001
- local epochs = 20

La migliore configurazione dopo 100 round di training è stata quella con **learning rate = 2.0** e ρ = **0.01**, che ha ottenuto valori delle metriche molto simili ai precedenti:

- Accuracy: 0.98
- Precision: 0.96
- Recall: 0.86

- F1-Score: 0.91
- Loss: 0.10

Confrontando le curve delle metriche al variare del numero di round notiamo inoltre che FedADMM diverge con valori di $\rho = 1.0$ e superiori (vedi figura 4.5).

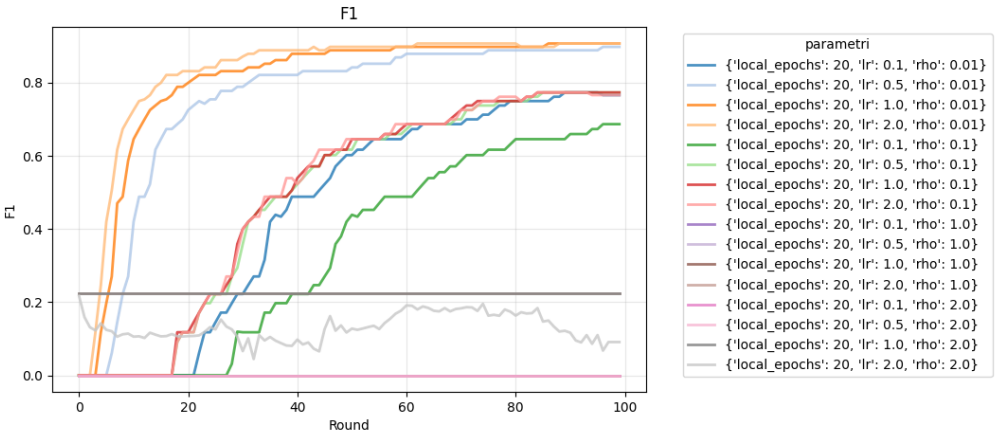


Figure 4.5: Andamento di F1 al variare dei round per l'algoritmo FedADMM

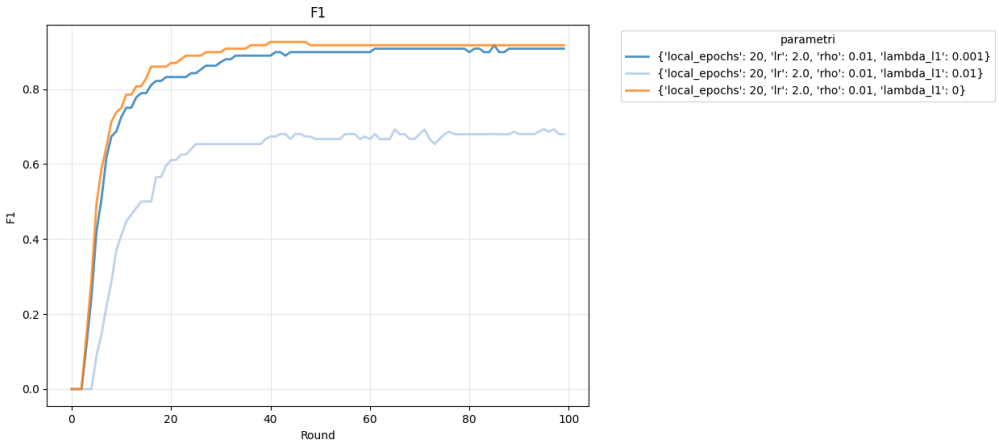


Figure 4.6: Andamento di F1 per l'algoritmo FedADMM al variare della regolarizzazione globale L1

Testiamo inoltre questa configurazione appena trovata al variare del parametro di sparsità globale λ L1 che permette di azzerare i pesi delle features non utili,

questo permette di ottenere una soluzione più sparsa e potenzialmente leggera da trasmettere al server.

Testando i valori λ L1 = [0.001, 0.01, 0] si nota che la F1 score scende di molto per $\lambda = 0.01$ mentre mantiene comunque un andamento simile per $\lambda = 0.001$ e $\lambda = 0$ (vedi figura 4.6), questo indica che è possibile applicare il parametro di regolarizzazione globale L1 ottenendo una soluzione più sparsa senza compromettere le performance del modello.

Confronto tra gli algoritmi

I tre algoritmi federati sono stati addestrati considerando uno scenario moderato di distribuzione Non-IID dei dati tra i client (alpha = 0.5) per trovare una configurazione degli iperparametri performante.

A questo punto per verificare la robustezza degli algoritmi a uno scenario reale verifichiamo come si comportano gli algoritmi al variare di alpha tra **100** (**distribuzione ottimale IID**) e **0.1** (**distribuzione fortemente Non-IID**).

Le configurazioni testate sono quelle che per ciascun algoritmo hanno ottenuto i risultati migliori sul validation set. In questo caso valutiamo le performance sul **test set** riservato all'inizio, che gli algoritmi non hanno mai visto.

- **FedAvg**: lr: 3.0, λ $L2$: 0.001,
- **FedProx**: lr: 2.0, μ: 0.001, λ $L2$: 0.001,
- **FedADMM**: lr: 2.0, ρ: 0.01, λ $L1$: 0.001, λ $L2$: 0.001

Gli altri parametri sono stati fissati per tutti e tre a: **rounds: 200**, **local epochs: 20**, **batch size: 64**.

Dall'analisi dei grafici di F1 score e loss sul test det dei tre algoritmi al variare di alpha, si osserva che in caso di distribuzione mediamente Non-IID (**alpha=0.5** e **alpha=1.0**) l'andamento dei tre algoritmi è simile attestandosi su valori di F1 = 0.95 dopo circa 100 round, vedi figura 4.7

Le differenze maggiori si sono riscontrate nei due scenari "estremi":

- per **alpha = 100**, quindi una distribuzione ottimale IID dei dati, FedAvg e FedProx raggiungono performance leggermente migliori (F1 = 0.95) rispetto a FedADMM (F1 = 0.91) con una velocità di convergenza maggiore, vedi figura 4.9
- per **alpha = 0.1**, quindi una distribuzione Non-IID estrema dei dati, FedADMM riesce ad mantenere risultati ottimi e nettamente migliori degli altri raggiungendo una F1 = 0.86 già dopo 20 round e raggiungendo un valore stabile di F1 = 0.89 dopo 100 round. Al contrario FedAvg e FedProx hanno una

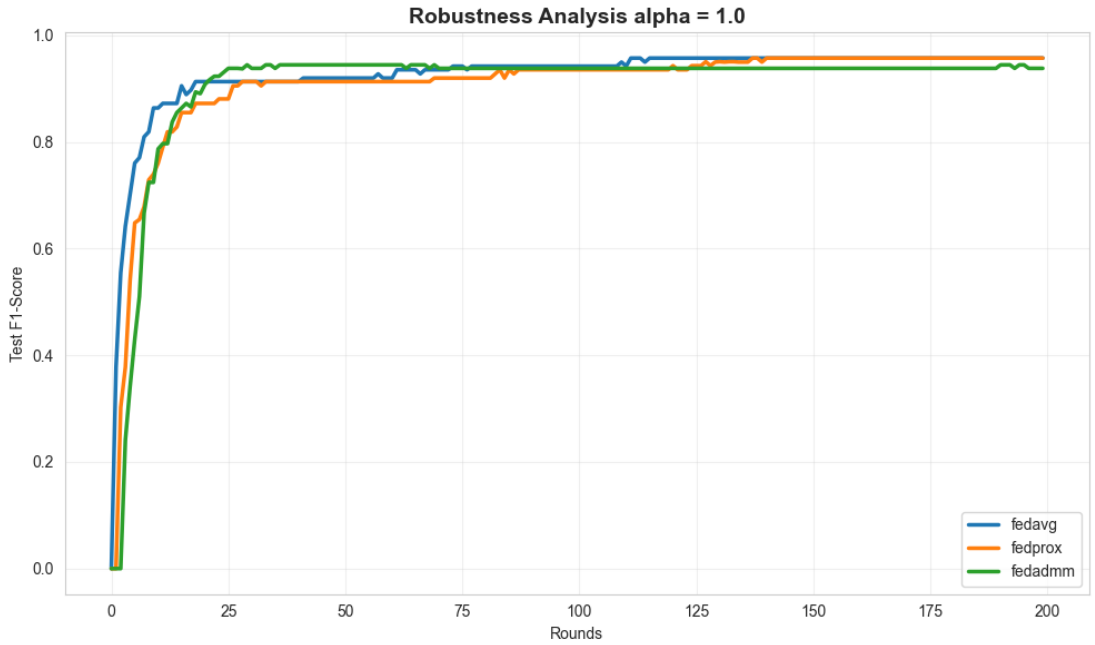


Figure 4.7: Confronto F1-score algoritmi per alpha=1.0

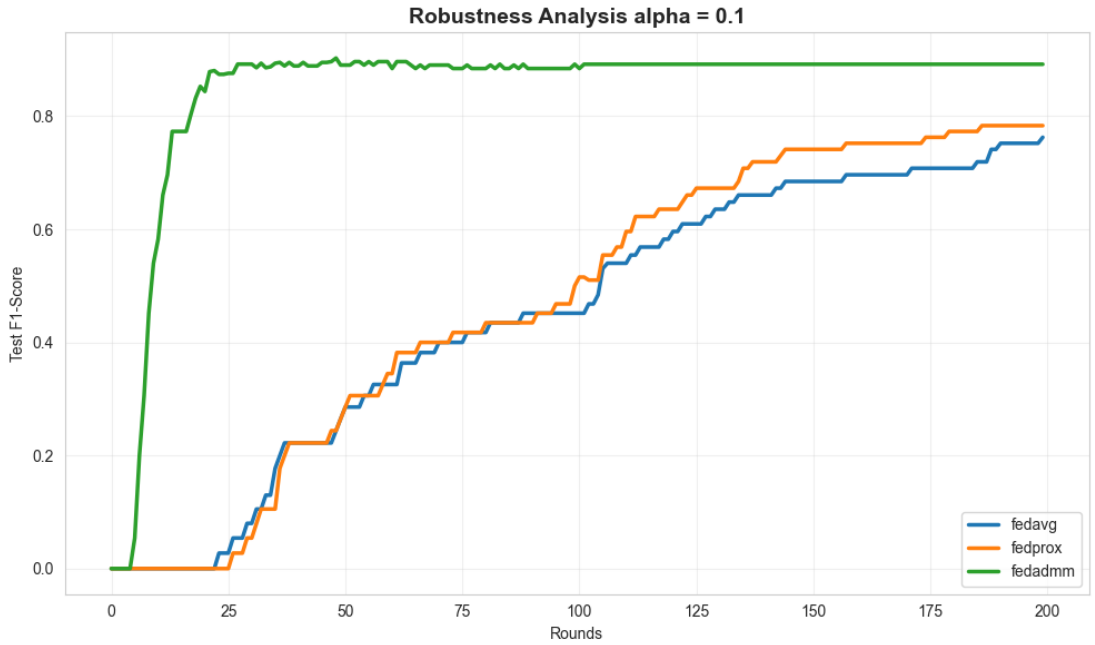


Figure 4.8: Confronto F1-score algoritmi in scenario Non-IID estremo (alpha=0.1)

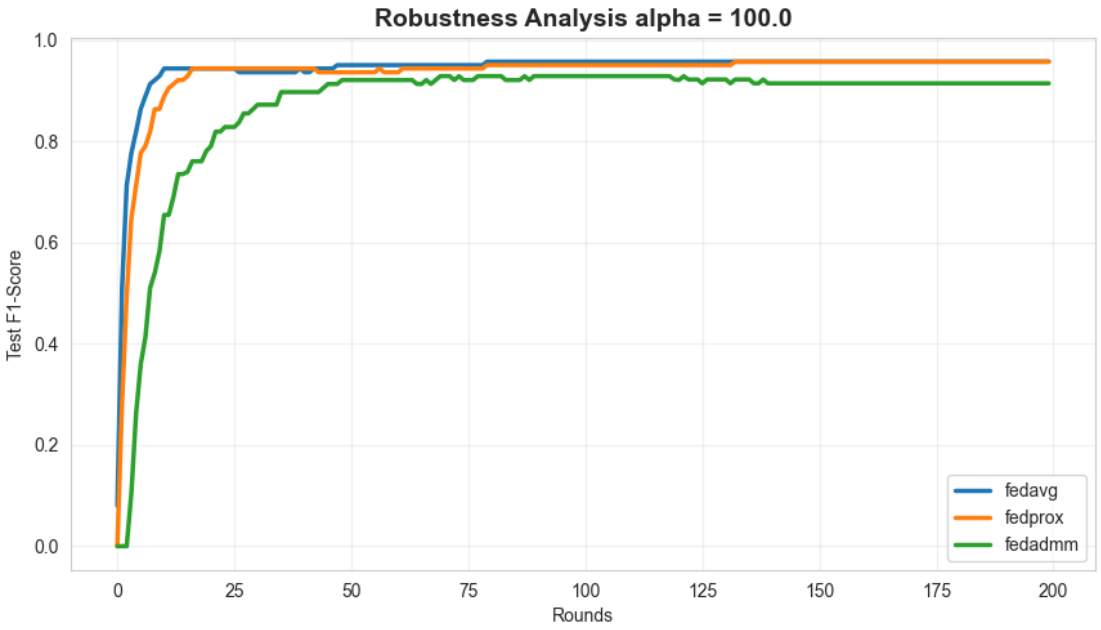


Figure 4.9: Confronto F1-score algoritmi in scenario IID (alpha=100)

convergenza molto più lenta raggiungendo solamente una F1 = 0.5 dopo 100 round, mentre dopo 200 round non riescono ancora a raggiungere le performance di ADMM, vedi figura 4.8.

Questo risultato mostra che la formulazione del problema di ottimizzazione tramite variabili duali con **ADMM** permette di ottenere buoni risultati di convergenza al consenso globale e una ottima robustezza in scenari Non-IID rispetto ad algoritmi più semplici. Inoltre, l'uso del parametro di regolarizzazione globale λ_{L1} ha permesso di ottenere modelli globali più sparsi, eliminando efficacemente le features rumorose senza degradare le performance.

Tuttavia **FedADMM** comporta alcuni svantaggi in contesti reali (ad es. dispositivi IoT o mobili): in particolare, differenza di FedAvg e FedProx che sono algoritmi *stateless* (il client scarica il modello, lo aggiorna e lo invia, senza dover memorizzare nulla), FedADMM richiede che ogni client mantenga in memoria locale il proprio stato tra un round e l'altro con le variabili duali u_k e l'ultimo modello locale w_k. Per modelli di grandi dimensioni come reti neurali, questo aspetto potrebbe risultare in un occupazione di memoria troppo elevata sui dispositivi.

Inoltre in contesti reali con un numero elevato di dispositivi, questi vengono di solito selezionati per l'addestramento in modo casuale tra un round e l'altro quindi non possono mantenere il proprio stato, di conseguenza in questi contesti un algoritmo più semplice come **FedProx** è sicuramente più adatto.

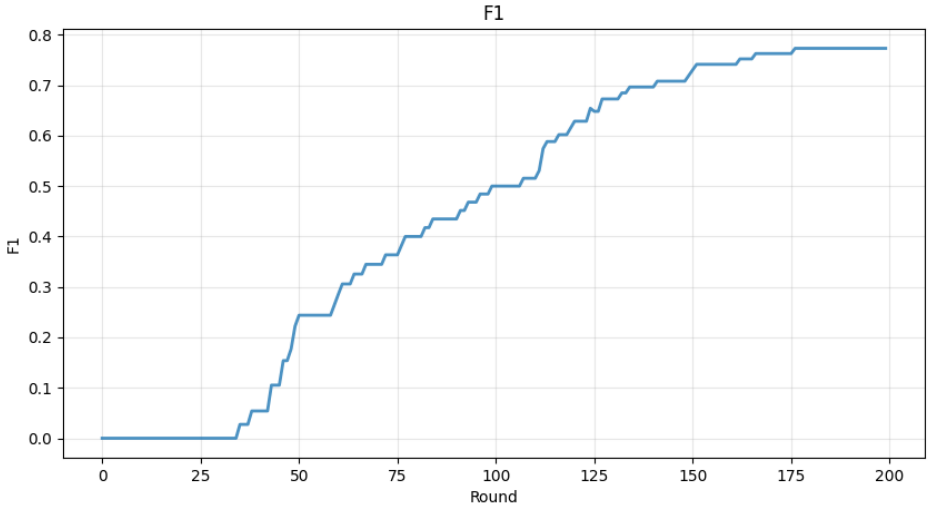


Figure 4.10: Andamento configurazione finale FedProx sul test set con alpha=0.1

Tuning di FedProx nel caso Non-IID estremo

Come ultimo test si esegue un nuovo fine tuning degli iperparametri di FedProx per vedere se c'è una configurazione che ottiene risultati migliori della precedente nello scenario di distribuzione Non-IID estrema (alpha=0.1).

Eseguendo la grid search sul validation set fissando sempre rounds: 200, local epochs: 20, batch size: 64 otteniamo che la configurazione migliore è data da **learning rate = 2.0** e $\mu = \mathbf{0.001}$ che ottiene performance leggermente migliori di quella precedente con stessa learning rate ma $\mu = 0.001$.

Eseguendo poi questa configurazione sul test set si ottiene l'andamento della F1-score visibile in figura 4.10, quindi il valore maggiore di μ non migliora comunque la velocità di convergenza dell'algoritmo FedProx in scenari fortemente Non-IID.

5 Bibliography

[1] Foundations and Trends in Machine Learning, Vol 4 Issue 1, 2021 - *Advances and Open Problems in Federated Learning* - `https://arxiv.org/abs/1912.04977`

[2] Hard, Rao et al., Google Research, 2019 - *Federated Learning for Mobile Keyboard prediction* - `https://arxiv.org/abs/1811.03604`

[3] 20th International Conference on Artificial Intelligence and Statistics (AISTATS) 2017 - *Communication-Efficient Learning of Deep Networks from Decentralized Data* - https://arxiv.org/abs/1602.05629

[4] Proceedings of the 3rd MLSys Conference, 2020 - *Federated Optimization in Heterogeneous Networks* - https://arxiv.org/abs/1812.06127